КРИПТОГРАФІЯ

КОМП’ЮТЕРНИЙ ПРАКТИКУМ №3

Криптоаналіз афінної біграмної підстановки

ФБ-43 Цвілій Влада

Варіант 23

Мета роботи

Набуття навичок частотного аналізу на прикладі розкриття моноалфавітної підстановки; опанування прийомами роботи в модулярній арифметиці.

Порядок виконання роботи

1. Реалізувати підпрограми із необхідними математичними операціями: обчисленням оберненого елементу за модулем із використанням розширеного алгоритму Евкліда, розв'язуванням лінійних порівнянь. При розв'язуванні порівнянь потрібно коректно обробляти випадок із декількома розв'язками, повертаючи їх усі.

2. За допомогою програми обчислення частот біграм, яка написана в ході виконання комп'ютерного практикуму №1, знайти 5 найчастіших біграм запропонованого шифртексту (за варіантом).

3. Перебрати можливі варіанти співставлення частих біграм мови та частих біграм шифртексту (розглядаючи пари біграм із п'яти найчастіших). Для кожного співставлення знайти можливі кандидати на ключ (a,b) шляхом розв'язання системи (1).

4. Для кожного кандидата на ключ дешифрувати шифртекст. Якщо шифртекст не є змістовним текстом російською мовою, відкинути цього кандидата. 5. Повторювати дії 3-4 дотi, доки дешифрований текст не буде змістовним.

Хід роботи

Аналіз афінного шифру та його слабкі місця, що експлуатуються в атаці

Афінний біграмний шифр є узагальненням класичного афінного шифру на випадок, коли базовим елементом відкритого тексту є не окрема літера, а блок із двох літер (біграма). Хоча розмір алфавіту підстановки при цьому збільшується з $\mathrm{m} = 31$ до $\mathrm{m}^2 = 961$, шифр залишається моноалфавітною підстановкою над множиною біграм. Це зумовлює його фундаментальні вразливості:

Збереження частотного профілю біграм: Якщо певна біграма відкритого тексту зустрічається в тексті часто, то зашифрована біграма, яка їй відповідає, матиме точно таку ж частоту появи в шифртексті на парних позиціях.

Лінійність перетворення: Зв'язок між відкритим та зашифрованим значенням є лінійним у кільці лишків. Це дозволяє за двома гіпотетичними відповідностями пар $(X^* \to Y^*)$ та $(X^{**} \to Y^{**})$ скласти систему лінійних порівнянь і легко виключити невідомий параметр b, звівши знаходження ключа до простого порівняння відносно a.

Малий простір ключів: Кількість можливих параметрів b дорівнює $m^2 = 961$. Параметр a повинен задовольняти умову $\gcd(a, 31) = 1$, отже, кількість можливих значень a дорівнює $961 - 31 = 930$. Загальний простір ключів становить $930 \times 961 = 893730$ варіантів. Навіть повний перебір на комп'ютері займає частки секунди, а аналітична атака за частотами біграм скорочує кількість кандидатів до кількох десятків чи сотень.

Методи, використані в коді, та метод автоматичного розпізнавання російського тексту

У програмній реалізації криптоаналізу було об'єднано теоретико-числові алгоритми та статистичні критерії:

- Розширений алгоритм Евкліда:

Використовується для знаходження найбільшого спільного дільника та оберненого елемента за модулем 961.

Необхідний як для розв'язання лінійних порівнянь, так і для процедури дешифрування (обчислення $a^{-1} \bmod 961$).

- Алгоритм розв'язання лінійних порівнянь:

Розв'язує порівняння $\Delta X \cdot a \equiv \Delta Y \pmod{m^2}$, де $\Delta X = (X^* - X^{**}) \bmod m^2$, $\Delta Y = (Y^* - Y^{**}) \bmod m^2$.

Коректно обробляє ситуацію, коли рівняння дає не один розв'язок, а одразу цілий список можливих ключів: якщо рівняння має кілька розв'язків, алгоритм повертає не одне число, а всі можливі варіанти ключа. Це гарантує, що ми не пропустимо справжній ключ шифрування під час перевірки.

- Метод автоматичного виявлення змістовного тексту:

Для автоматичної перевірки знайдених ключів замість ручного перегляду було використано індекс відповідності. Якщо текст є випадковим набором символів, усі літери зустрічаються приблизно з однаковою частотою, а якщо це природна мова - деякі літери зустрічаються набагато частіше за інші.

На відміну від словників чи списків заборонених слів, індекс відповідності не вимагає завантаження додаткових баз даних і не ламається через граматичні помилки чи рідкісні слова.

Для випадкового сміттєвого тексту з 31 літери значення I становить приблизно 0.032, а для нормального змістовного тексту російської мови значення I становить приблизно 0.053 - 0.057.

У програмі встановлено поріг I > 0.048 для аналізу перших 600 символів дешифрованого фрагмента. Оскільки між шумом і реальною мовою є великий розрив, цей поріг миттєво відсікає всі хибні ключі та зупиняє пошук на правильному тексті.

Можливі проблеми та шляхи їх розв'язання

Випадок, коли різниця біграм ділиться на 31:

Якщо різниця числових значень обраної пари біграм ділиться на 31, то знайти обернене число звичайним шляхом неможливо, тому що різниця і модуль 961 мають спільний дільник 31. Через це просте лінійне порівняння не мало б одного стандартного розв'язку. Тому у коді реалізовано алгоритм розв'язання лінійних порівнянь, який у такому випадку ділить усе рівняння на 31 і знаходить не один, а одразу всі 31 можливий варіант для ключа a. Ті варіанти, які самі діляться на 31, відкидаються перевіркою gcd(a, 31) == 1, а решта кандидатів перевіряється дешифруванням.

Невідповідність отриманого топу біграм у шифртексті еталонному топу мови:

У конкретному фрагменті художнього або публіцистичного тексту частоти біграм можуть відхилятися від загальномовних статистичних таблиць. Наприклад, біграма ст, яка є найчастішою у мові загалом, у даному тексті могла посісти 3 або 4 місце. Тому замість жорсткого зіставлення лише пари найбільш частих елементів було реалізовано перебір між розширеним списком біграм мови (топ-15) та топ-10 біграмами шифртексту.

Сміттєві символи та невидимі артефакти кодування в шифртексті:

У вхідних даних були присутні переноси рядків, пробіли, а також зустрічалася літера «і», відсутня в 31-символьному російському алфавіті. Це могло збити парність довжини тексту або викликати аварійне завершення через помилку індексації. Тому введено блок попередньої очистки тексту: приведення до

нижнього регістру, автоматична заміна ё -> е, ъ -> ь, і -> и, видалення неалфавітних знаків та гарантування парної довжини рядка.

Написання програми

Програма зламує шифр методом атаки за відомим шифртекстом, поєднуючи частотний аналіз пар літер і швидку перевірку змістовності через індекс відповідності.

Робота виконується в кілька кроків:

- Підготовка тексту: шифртекст переводиться у нижній регістр. Літера "ё" замінюється на "е", літера "ъ" на "ь", а випадкові літери іншої розкладки замінюються на "и". Видаляються пробіли та переноси рядків, а якщо довжина тексту виявилася непарною, останній зайвий символ відкидається.
- Підрахунок частот: шифртекст ріжеться на окремі пари літер, що не перекриваються. Програма рахує, скільки разів зустрічається кожна пара, і вибирає топ найчастіших біграм шифртексту.
- Складання гіпотез: беруться дві часті біграми з таблиці мови та дві часті біграми з шифртексту. Програма припускає, що відкриті біграми перетворилися саме на ці шифровані.
- Обчислення ключів a і b: для кожної пари гіпотез програма розв'язує лінійне рівняння з остачею від ділення на 961. Звідти знаходяться всі можливі значення ключа a. Ті значення a, які діляться на 31, одразу відсіюються, бо для них розшифрування неможливе. Для правильних значень a обчислюється другий ключ - зсув b.
- Автоматичний фільтр змістовності: отриманим ключем (a, b) розшифровується тільки пробний шматочок тексту на 600 літер. Програма рахує для нього показник I: якщо I менший або дорівнює 0.048 - це беззмістовний текст, ключ відкидається, а якщо I більший за 0.048 - знайдено справжній змістовний текст, а ключ є правильним.
- Як тільки правильний ключ знайдено, програма розшифровує вже весь текст повністю, виводить результати на екран і зберігає розшифрований файл на диск.



```python
import collections
import math

# --- 1. Алфавіт (31 літера) ---
ALPHABET = "абвгдежзийклмнопрстуфхцчшщьыэюя"
M = len(ALPHABET)  # 31
M2 = M * M  # 961

CHAR_TO_IDX = {ch: i for i, ch in enumerate(ALPHABET)}
IDX_TO_CHAR = {i: ch for i, ch in enumerate(ALPHABET)}

# Розширений топ найчастіших біграм російської мови
RUS_TOP_BIGRAMS = [
    "то",
    "ст",
    "но",
    "на",
    "ен",
    "не",
    "ов",
    "ра",
    "во",
    "ко",
    "ро",
    "по",
    "ос",
    "ни",
    "ер",
]
```

Початкове налаштування алфавіту та статистичних даних:

Задається робочий алфавіт із 31 літери, визначається його розмір 31 та розмір простору біграм 961.

Створюються словники CHAR_TO_IDX та IDX_TO_CHAR для швидкого переведення літер у їхні порядкові номери і навпаки.

Задається розширений список найчастіших біграм мови (RUS_TOP_BIGRAMS) із 15 пар, який використовується як еталон для частотного аналізу.

```python

# --- 2. Математика ---
def ext_gcd(a: int, b: int):
    if b == 0:
        return a, 1, 0
    d, u1, v1 = ext_gcd(b, a % b)
    return d, v1, u1 - (a // b) * v1


def mod_inverse(a: int, n: int):
    d, u, _ = ext_gcd(a, n)
    if d != 1:
        return None
    return (u % n + n) % n


def solve_linear_congruence(a: int, b: int, n: int):
    a = (a % n + n) % n
    b = (b % n + n) % n
    d, u, _ = ext_gcd(a, n)
    if b % d != 0:
        return []
    a1, b1, n1 = a // d, b // d, n // d
    inv_a1 = mod_inverse(a1, n1)
    if inv_a1 is None:
        return []
    x0 = (b1 * inv_a1) % n1
    return [(x0 + i * n1) % n for i in range(d)]

```

Функція ext_gcd реалізує розширений алгоритм Евкліда, знаходячи найбільший спільний дільник двох чисел та коефіцієнти для пошуку оберненого елемента.

Функція mod_inverse знаходить обернене число за заданим модулем або повертає None, якщо числа не є взаємно простими.

Функція solve_linear_congruence знаходить усі корені лінійного рівняння з остачею. Якщо рівняння має кілька розв'язків, вона коректно генерує і повертає повний список усіх можливих значень кореня, щоб не втратити справжній ключ.

```python

# --- 3. Біграми та Дешифрування ---
def bigram_to_num(bg: str) -> int:
    return CHAR_TO_IDX[bg[0]] * M + CHAR_TO_IDX[bg[1]]


def num_to_bigram(val: int) -> str:
    return IDX_TO_CHAR[val // M] + IDX_TO_CHAR[val % M]


def decrypt_text(ciphertext: str, a: int, b: int) -> str:
    inv_a = mod_inverse(a, M2)
    if inv_a is None:
        return ""
    res = []
    for i in range(0, len(ciphertext) - 1, 2):
        y = bigram_to_num(ciphertext[i : i + 2])
        x = (inv_a * (y - b)) % M2
        res.append(num_to_bigram(x))
    return "".join(res)

```

Функція bigram_to_num склеює пару літер в одне число від 0 до 960.

Функція num_to_bigram розпаковує число назад у дві літери через цілу частину від ділення на 31 (перша літера) та остачу (друга літера).

Функція decrypt_text виконує розшифрування тексту за знайденими ключами a і b: спочатку знаходить обернений ключ a за модулем 961, а потім по черзі бере пари шифртексту, віднімає зсув b, множить на обернений ключ і повертає відкритий текст.

```python

# --- 4. Індекс відповідності ---
def index_of_coincidence(text: str) -> float:
    n = len(text)
    if n <= 1:
        return 0.0
    counts = collections.Counter(text)
    return sum(c * (c - 1) for c in counts.values()) / (n * (n - 1))


# --- 5. Основний блок ---
def main():
    raw_ct = """
    дплргшрщзчфуилшччьолрхсжфпцнбтйяэзьзншпзвлплщуфонубинптщьаьчжлсзбцдщзчпжйргбсхфэчвездусззлгудудуэедх
    тярхездуежшзфхшуфхцппшлхтрфхюхдпшгташкнжрэокфхшруздуушлушудойгтфдхуртзлефунптщзчжлдщвзоюэтфхфлаэзону
    згтфйинпшэеьщлрбнладнптщзчщуьлдщзчшгуомээаээээябрипьеуггбилцчбжрщэнзуфонубиязйэщйбиэхфчугвздуокйрыю
    бдбжикамэрцэьгфхнхзшщюбжллжзщэйхщхзлмхизязеэцэпжлряхщхбрьгбрцэтщфпяхпхвуобзэргллрчугвздуфущухлфетжэр
    рэоьейпжнуежлонусуфуцарголжчцлжбккеяолбтплщущэвмфхнджбглюлпблющзгнчоргзчдозэргфуруюдрхыедхфудорплойя
    окзчфхчявчшжочбяелуэпжхпзядхсехескпзспщйбжсзтэрщаоргллрчугзягужблтжбэрзлчзлгуофхяхщщипугдвпладеьчрвм
    ьруозчгунмбяпвокрщеэряшлиршкшлбруоежжзядоюшэьчсздхфоглупшгчоцхоюьчышйькхдхамфжуобтдхюххескйрзунхюлфх
    цупонубтргфужбзэщзнубжрхолырвмьрсодуькйрээялжэбжщюуосоумьчюхлжжлнучэрщбтргщхчьяойзплцупрзэккпзнуожщю
```

Функція index_of_coincidence обчислює індекс відповідності для переданого тексту на основі частот появи кожного символу.

```
    рхфуюзеэюшнжуонлцчисшуэжьчэзьухюзуфьщувурбуоабффхмлядйяргагьчжлдщллборщтфюшфпьуцхоэрщеэташупэрщнжзб
    пжзчгуцчрлхкшлльозюггушуйьэудакгдонуцхгчумвкюзлзоэфжокшлсздощздуюгипушнжикйрьляергипушнжшчухдоеийх
    """
    cleaned = (
        raw_ct.lower()
        .replace("ё", "е")
        .replace("ъ", "ь")
        .replace("і", "и")
        .replace("ї", "и")
    )
    ct = "".join(ch for ch in cleaned if ch in CHAR_TO_IDX)
    if len(ct) % 2 != 0:
        ct = ct[:-1]

    # Беремо топ-10 неперетинних біграм шифртексту
    non_overlap_bigrams = [ct[i : i + 2] for i in range(0, len(ct), 2)]
    counts = collections.Counter(non_overlap_bigrams)
    top_ct_bigrams = [item[0] for item in counts.most_common(10)]

    print("=== ЧАСТОТНИЙ АНАЛІЗ ШИФРТЕКСТУ ===")
    print(f"Загальна кількість біграм: {len(non_overlap_bigrams)}")
    print("Топ-10 біграм шифртексту:")
    for bg, c in counts.most_common(10):
        print(f"  '{bg}': {c}")

    top_ct_nums = [bigram_to_num(bg) for bg in top_ct_bigrams]
    top_rus_nums = [bigram_to_num(bg) for bg in RUS_TOP_BIGRAMS]

    print("\n=== ПОШУК КЛЮЧА ТА ДЕШИФРУВАННЯ ===")
    tested_keys = set()
    best_candidate = None
    max_ic = 0.0
```

Текст очищується від пробілів і переносів рядків, літери замінюються за вимогами завдання ("ё" на "е", "ъ" на "ь", випадкові літери "і/ї" на "и"), а довжина тексту робиться парною.

Очищений шифртекст розбивається на пари літер. За допомогою collections підраховуються частоти їх появи та виділяється топ-10 найчастіших біграм шифртексту.

Знайдені часті біграми шифру та еталонні біграми мови переводяться у числові значення для подальших розрахунків. tested_keys треба, щоб уникнути повторної перевірки однакових ключів.

```
183         # Перебираємо комбінації
184         for i in range(len(RUS_TOP_BIGRAMS)):
185             for j in range(len(RUS_TOP_BIGRAMS)):
186                 if i == j:
187                     continue
188                 x1, x2 = top_rus_nums[i], top_rus_nums[j]
189                 dx = (x1 - x2) % M2
190 
191                 for k in range(len(top_ct_bigrams)):
192                     for l in range(len(top_ct_bigrams)):
193                         if k == l:
194                             continue
195                         y1, y2 = top_ct_nums[k], top_ct_nums[l]
196                         dy = (y1 - y2) % M2
197 
198                         possible_a = solve_linear_congruence(dx, dy, M2)
199                         for a in possible_a:
200                             if math.gcd(a, M) != 1:
201                                 continue
202                             b = (y1 - a * x1) % M2
203                             if (a, b) in tested_keys:
204                                 continue
205                             tested_keys.add((a, b))
206 
207                             # Тестуємо перші 600 символів на Індекс Відповідності
208                             sample_dec = decrypt_text(ct[:600], a, b)
209                             ic = index_of_coincidence(sample_dec)
210 
211                             if ic > max_ic:
212                                 max_ic = ic
213 
```

Програма бере дві різні біграми мови і дві різні біграми шифртексту, обчислює різниці між ними та розв'язує лінійне рівняння для пошуку ключа a. Знайдені значення перевіряються на взаємну простоту з числом 31, після чого для валідних варіантів обчислюється параметр b. Отримана пара ключів додається до списку перевірених, щоб уникнути повторів. Далі знайденим ключем розшифровується тестовий фрагмент тексту довжиною 600 символів і для нього розраховується індекс відповідності I, значення якого оновлює поточний максимум.

```
                    # Якщо IC > 0.048 - це 100% російська мова (у шуму ~0.032)
                    if ic > 0.048:
                        full_dec = decrypt_text(ct, a, b)
                        print(
                            f"\n[+] УСПІХ! ЗНАЙДЕНО КЛЮЧ: a = {a}, b = {b}"
                        )
                        print(
                            f"Індекс відповідності (IC): {ic:.5f} (норма мови)"
                        )
                        print(
                            f"Зіставлення: '{RUS_TOP_BIGRAMS[i]}' -> '{top_ct_bigrams[k]}', '{RUS_TOP_BIGRAMS[j]}' -> '{top_ct_bigrams[l]}'"
                        )
                        print(
                            f"\nПочаток розшифрованого тексту:\n{full_dec[:400]}\n"
                        )

                        with open(
                            "decrypted_lab3.txt", "w", encoding="utf-8"
                        ) as f:
                            f.write(full_dec)
                        print(
                            "Повний розшифрований текст збережено у 'decrypted_lab3.txt'"
                        )
                        return

    print(f"[-] Не знайдено. Максимальний знайдений IC був: {max_ic:.5f}")


if __name__ == "__main__":
    main()
```

Якщо порахований індекс відповідності перевищує поріг 0.048, програма вважає ключ знайденим, оскільки такий показник притаманний лише змістовній мові. У цьому випадку шифртекст повністю дешифрується знайденою парою ключів a і b. Програма виводить значення знайдених ключів, показник індексу відповідності, пари біграм, які привели до успіху, та перші рядки прочитаного тексту. Повний розшифрований текст автоматично зберігається у файл decrypted_lab3.txt, після чого робота функції завершується. Якщо жодна комбінація не підійшла, в кінці виводиться максимальне зафіксоване значення індексу відповідності.

Результат:

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Lenovo\Desktop\ФБ43_Цвілій\3> & C:\Users\Lenovo\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/Leno
=== ЧАСТОТНИЙ АНАЛІЗ ШИФРТЕКСТУ ===
Загальна кількість біграм: 2899
Топ-10 біграм шифртексту:
  'до': 56
  'ну': 49
  'рх': 47
  'шу': 40
  'шг': 39
  'фх': 37
  'жб': 35
  'зч': 33
  'ао': 32
  'фу': 30

=== ПОШУК КЛЮЧА ТА ДЕШИФРУВАННЯ ===

[+] УСПІХ! ЗНАЙДЕНО КЛЮЧ: a = 735, b = 300
Індекс відповідності (IC): 0.05273 (норма мови)
Зіставлення: 'то' -> 'шу', 'ст' -> 'до'

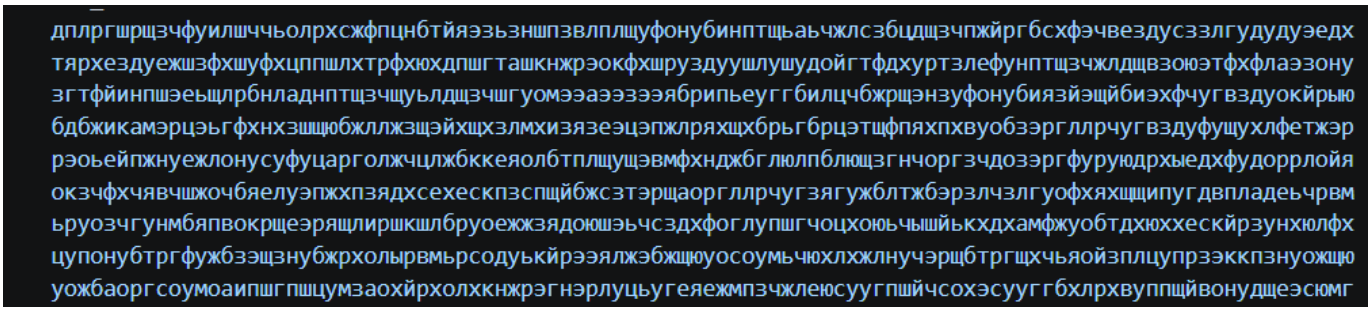
Початок розшифрованого тексту:
владькогосподиусьфшинасмолящихсятещеукреписилоютвоеюблагзчюстивейшегосамодержавнейшеговеликогогосударянашегоимператорааалекса
ротостывоздаждыемуподлагостиегоеюжехранитньтвойвозлюбленюфйизраилыблагослоыиегосоветьначинанияиделшутвердивсемогущноютвоеюде
едуншврагаякожемоисеюнаамаликагедеонунамадиамаидшвидунаголиафасохранивоинствоегоположилукмед
```

П'ять найчастіших біграм досліджуваного шифртексту:

- до (56 разів)
- ну (49 разів)
- рх (47 разів)
- шу (40 разів)
- шг (39 разів)

Знайдений ключ шифрування: a = 735, b = 300.
Індекс відповідності розшифрованого тексту: 0.05273.
Пара біграм, яка привела до успішного зламу: відкрита біграма "то" перейшла в шифровану "шу", а відкрита біграма "ст" - у шифровану "до".

ШТ:



ВТ:

владькогосподиусьфшинасмолящихсятещеукреписилоютвоеюблагзчюстивейшегосамодержавнейшеговеликогогосударянашегоимператоралександрапавловичап
омяниправдуегоикротостывоздаждыемупоблагостиегоеюжехранитньтвойвозлюбленюфйиэраилыблагослоыиегосоветьначинанияиделшутвердивсемогущноютвоею
десницеюцарствоегоиподзждыемупобедуншврагаякожемоисеюнаамаликагедеонунамадиамаидшвидунаголиафасохранивоинствоегоположилукмедянэфшцамвоимят
воеополчившихсяипрепояшиихсилоюнабраныприимиоружиеищитивосстанивпомощынааудапостьдятсяипосрамятсямьслящийнамзлаядабудутпредлицемверноготив
оинстваякопрахпредлицемветраиангелтвойсилыюфйдабмдетоскорбляяйипогоняяйихдаприидетимсетыюженюсведаютиихловитваюжюсхкбфшадаобьметихдападутп
одногамирабовтвоихивпопраниевоемнашиудабмдутгокподинеизнеможетутебюспасативомногихивмаьфхтьесибогданепревозможетпротивутебечеловекбожеотец
нашихпомянищедрогфтвояимилостияжеотвекасутынеотвержинасотлицатвоегонижевозгнушайсянеэостоинствомнашимнопомилуйнакповелицеймилоститвоейипом
ножествубедроттвоихпрезрибеззаконияигрехинашасердцечистосозиждивнасидухправобновивоутробенашейвсехнасукрепиверояювтяутвердинадеждоюодушевии
стиннояюдругкодругулюбоыиювооружиединодушиемнаправедноезащибениеодержанияежедалюсинамиотцемнашиуданевознесетсяжезлнечюститфхнзжребийосвящен
юфхгосподибоженаавнегожеверуеминанегожеуповаемнепосраминасотчаяниямилоститвоеяисотворизнамениевоблагоякодаыидятненшвидящпйнасипршвославную
верунаауипосрамятсяипогибнутидшуведятвсестраюфякоимятещегокподвиэфлюдиетвоияыинамгосподиюфнемилостытвоюикпасениетвоедаждынамвозвюселисердц

Контрольні питання

1. Які статистичні властивості шифртексту були використані під час формування кандидатів на ключ?

Було використано властивість збереження частот появи неперетинних біграм. Оскільки афінний біграмний шифр є моноалфавітною підстановкою над блоками з двох літер, найчастіші біграми відкритого тексту переходять у найчастіші біграми шифртексту на тих самих парних позиціях. Підрахувавши абсолютні частоти неперетинних пар символів у шифртексті, ми виділили топ найбільш уживаних біграм та зіставили їх із теоретичним топом біграм російської мови.

2. Чому не кожне зіставлення частих біграм відкритого тексту та шифртексту приводить до допустимого ключа?

По-перше, лінійне рівняння для пошуку ключа a має розв'язки лише тоді, коли різниця шифрованих біграм ділиться на найбільший спільний дільник різниці відкритих біграм та модуля 961. Якщо дільник дорівнює 31, а різниця шифрованих біграм на 31 не ділиться, рівняння взагалі не має математичних розв'язків. По-друге, навіть коли рівняння має розв'язок, отримане значення ключа a може виявитися кратним числу 31. За правилами афінного шифру ключ a обов'язково має бути взаємно простим із розміром алфавіту 31, інакше не існує оберненого елемента за модулем 961 і текст принципово неможливо розшифрувати. По-третє, саме зіставлення біграм може бути помилковим через природні коливання частот у конкретному тексті.

3. Чи виникали під час криптоаналізу випадки, коли одне зіставлення біграм давало декілька кандидатів на ключ? Як такі кандидати оброблялися?

Так, такі випадки виникали. Це трапляється тоді, коли різниця числових значень пари відкритих біграм ділиться на 31, тобто їхній найбільший спільний дільник із числом 961 дорівнює 31. За умови існування розв'язку таке рівняння видає не одне число, а одразу 31 можливий корінь за модулем 961.

Усі такі кандидати оброблялися програмно: функція пошуку розв'язків повертала повний список із 31 значення. Потім кожне значення проходило фільтрацію на взаємну простоту з числом 31, а для коректних значень a обчислювався зсув b, після чого кожен отриманий ключ перевірявся пробним дешифруванням.

4. За якими ознаками серед отриманих кандидатів було визначено правильний ключ?

Правильний ключ було визначено за числовим значенням індексу відповідності дешифрованого фрагмента та візуальною змістовністю отриманого тексту. Для хибних ключів розшифрований текст виглядав як хаотичний набір літер із низьким індексом відповідності, тоді як для правильного ключа a = 735, b = 300 значення індексу зросло до 0.05273, що повністю відповідає нормі природної мови, а сам текст став осмисленим і легким для читання.

5. Який критерій автоматичного розпізнавання осмисленого тексту було використано? Чому було обрано саме його?

Було використано ентропійний критерій - індекс відповідності. Його обрано через високу надійність та простоту обчислення: він не потребує підключення

зовнішніх словників, списків заборонених слів чи морфологічного аналізу і не дає збоїв на текстах зі старовинною або специфічною лексикою. Між індексом випадкового шуму та індексом природної мови існує великий статистичний розрив, що робить цей критерій ідеальним автоматичним фільтром.

6. Як поводився обраний критерій для правильного відкритого тексту та для результатів розшифрування неправильними ключами?

Для неправильних ключів результат розшифрування являв собою рівномірний випадковий шум, тому показник індексу відповідності коливався біля теоретичного значення 0.032. При підстановці правильного ключа частотний розподіл літер миттєво відновився до мовної норми, і індекс відповідності підскочив до значення 0.05273. Це значно перевищило встановлений у коді поріг 0.048 і дозволило однозначно зафіксувати успішний злам без хибних спрацьовувань.

Висновок

Під час виконання лабораторної роботи було досліджено криптографічну стійкість афінного біграмного шифру та реалізовано атаку на основі частотного аналізу за відомим шифртекстом. Було опановано прийоми модулярної арифметики, зокрема розширений алгоритм Евкліда та алгоритм знаходження всіх розв'язків лінійних порівнянь у кільці лишків за модулем 961. Складено та розв'язано систему рівнянь на основі зіставлення найчастіших біграм шифртексту та еталонних біграм російської мови. Для автоматизації пошуку ключа було успішно застосовано критерій індексу відповідності, який забезпечив швидке та безпомилкове відсіювання випадкового шуму від змістовного тексту. Було знайдено правильний ключ шифрування a = 735, b = 300, на якому індекс відповідності досяг значення 0.05273, що дозволило повністю відновити вихідний відкритий текст.